

On Classical Graph Algorithms

DELIEN DELIEN (delien@aims.ac.za)
African Institute for Mathematical Sciences (AIMS)

Supervised by: Dr. Audace A. V. Dossou-Olory (University of Abomey-Calavi, Benin)
and
Dr. Sarah J. Selkirk (University of Warwick, UK)

16 June 2024

Submitted in partial fulfilment of a structured master's degree at AIMS South Africa

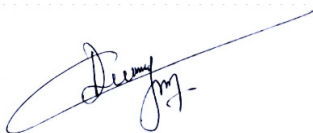


Abstract

This essay aims to describe some important results and classical algorithms in combinatorial optimization. The focus will be on some classical problems that can be formulated naturally in terms of graphs, for example, minimum spanning trees, shortest paths, and network flows. In this project, we will provide implementations of these algorithms in Python programming language, discuss the testing and validation of the implemented algorithms, and present the results obtained from applying the algorithms to a set of different graphs.

Declaration

I, the undersigned, hereby declare that the work contained in this research project is my original work and that any work done by others or by myself previously has been acknowledged and referenced accordingly.

.....

.....

DELIEN, 16 June 2024

Contents

Abstract	i
1 Introduction	1
1.1 Basics of Graphs	1
1.2 Types of Graphs	2
1.3 Properties of Graphs	4
1.4 Special Graphs	10
2 Optimisation	12
2.1 Graph Optimization Problems	12
2.2 Algorithms for Graph Optimization	14
3 Basic Algorithms	16
3.1 Algorithms for Minimum Spanning Trees (MST)	16
3.2 Algorithms for Single-source Shortest Path Problem (SSPP)	17
4 Network flows	22
4.1 Maximum Flow Problem	22
4.2 Minimum Cut Problem	25
5 Implementation of the algorithm in Python, and comparison of the results from Google Maps	27
5.1 Algorithms Implementation	27
5.2 Comparison with Google Maps	30
5.3 Performance Analysis	32
6 Conclusion	33
References	35

1. Introduction

1.1 Basics of Graphs

Graphs are one of the unifying themes of abstract representation in computer science that can describe interactions between humans, transportation systems, and telecommunications. The programmer needs to be able to model many different structures using a single formulation (Bleiel, 2019).

Graphs which are a fundamental combinatorial structure are used throughout this project. In this chapter, we review some definitions, characteristics, and types of graphs, and prove some basic theorems.

A graph was invented by Leonhard Euler in 1736 when he wanted to solve the problem of the “Seven Bridges of Königsberg”. He wanted to visit all four areas of a city that was connected by seven bridges by only crossing each bridge once (Korte and Vygen, 2007).

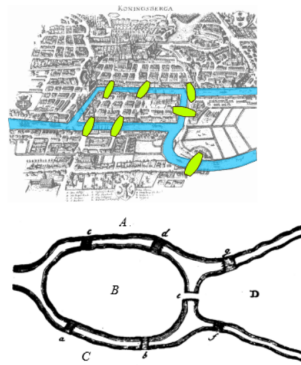


Figure 1.1: The seven bridges of Königsberg.

Figure 1.1 shows the diagram of the problem that Leonhard Euler wanted to solve.

1.1.1 Definition. A graph is a pair $G = (V, E)$, where V is called the set of vertices or nodes, and E is called the set of edges, such that E is a subset of $V \times V$.

1.1.2 Example. Figure 1.2 shows a graph that has set of edges $E = \{\{A, B\}, \{A, D\}, \{B, C\}, \{B, D\}, \{C, D\}\}$ and set of vertices $V = \{A, B, C, D\}$.

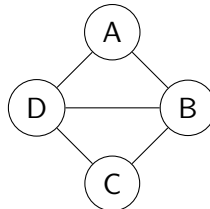


Figure 1.2: A graph with 4 vertices and 5 edges.

1.2 Types of Graphs

In graph theory, there are many types of graphs. In this section, we provide basic definitions for the given graphs.

1.2.1 Directed Graph or Digraph

1.2.2 Definition. A directed graph or digraph is a graph where all edges have an orientation: an edge from A to B is denoted (A, B) , as opposed to the undirected $\{A, B\}$ (Chartrand and Lesniak, 1979).

1.2.3 Example. An example of a directed graph is given in Figure 1.3.

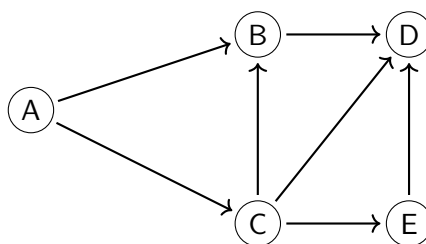


Figure 1.3: A directed graph on 5 vertices with 7 edges.

In addition, we need to define “in-degree” and “out-degree”.

1.2.4 Definition. In-degree is the number of edges directed to a vertex, and out-degree is the number of edges directed from a vertex.

1.2.5 Example. In Figure 1.3, the in-degree to vertex D is 3 and the out-degree from vertex A is 2.

1.2.6 Undirected Graph

1.2.7 Definition. An undirected graph is a graph that has edges and no orientation. The edge (B, A) is identical to the edge (A, B) (Chartrand and Lesniak, 1979).

1.2.8 Example. There is an example of an undirected graph in Figure 1.4.

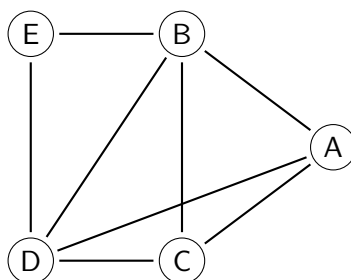


Figure 1.4: An undirected graph on 5 vertices.

1.2.9 Weighted Graphs

1.2.10 Definition. A weighted graph is a graph in which the edges of the graph are assigned an additional parameter of a number to indicate a weight.

Moreover, a weighted graph is characterized by negative and positive edge weights.

1.2.11 Example. Here are examples of the graphs that have positive or negative edge weights.

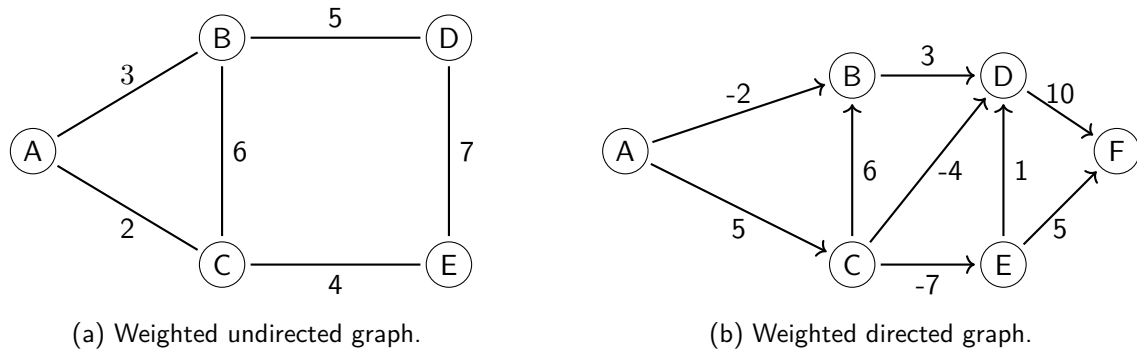


Figure 1.5: (a) contains the positive weights, and (b) contains both negative and positive weights.

1.2.12 Degree of vertex

1.2.13 Definition. The degree of a vertex is the number of edges attached to each vertex.

1.2.14 Example. Let G be a graph that has vertices with odd and even degrees, see Figure 1.6 for an example.

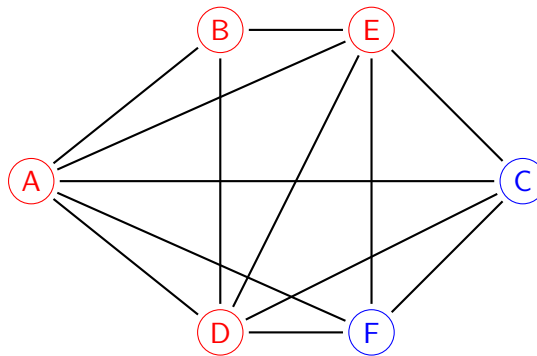


Figure 1.6: Red (odd) and blue (even) vertices.

(A): degree 5, (B): degree 3, (C): degree 4, (D): degree 5, (E): degree 5, (F): degree 4.

1.2.15 Regular Graph

1.2.16 Definition. A regular graph is a graph in which all vertices have the same degree. If the degree of all vertices in the graph is k , then we call this a ' k -regular graph'.

1.2.17 Example. Here is an example of a regular graph.

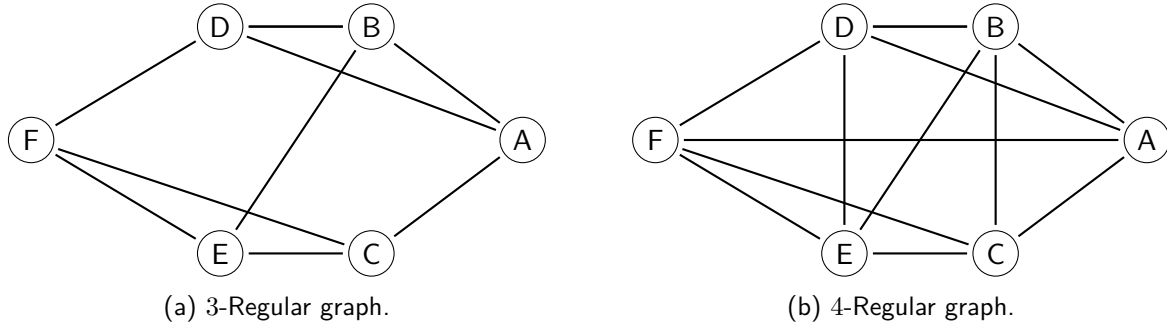


Figure 1.7: In **(a)**, each vertex has degree 3, while in **(b)**, every vertex has degree 4. So we can say that **(a)** is a 3-regular graph and **(b)** is a 4-regular graph.

1.3 Properties of Graphs

1.3.1 Complete Graph

1.3.2 Definition. Let $G = (V, E)$ be a graph. G is considered complete if and only if every pair of distinct vertices is joined by an edge (Bondy and Murty, 1976).

1.3.3 Lemma. We denote by K_n a complete graph on n vertices.

1. K_n as an undirected graph contains exactly $\frac{n(n-1)}{2}$ edges.
2. In a directed graph, it contains $n(n-1)$ edges.

Proof. 1. An undirected complete graph that has n vertices contains exactly $\frac{n(n-1)}{2}$ edges. In a complete undirected graph, each pair of distinct vertices forms an edge, and the total number of edges according to this reason can be written a binomial form such that $|E| = \binom{n}{2}$.

$$|E| = \binom{n}{2} = \frac{n(n-1)}{2}.$$

Hence, the number of edges in an undirected graph that has n vertices is exactly

$$|E| = \frac{n(n-1)}{2}.$$

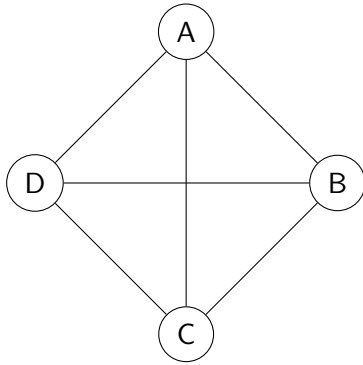
2. A complete directed graph that has n vertices contains $n(n-1)$ edges. Let G be a complete directed graph with n vertices. Consider $v_i, 1 \leq i \leq n$ and v_j are vertex with $i \neq j$ such that each vertex v_i has a directed edge to every vertex v_j . Each vertex has $n-1$ outgoing edges.

A directed graph with n vertices has several edges

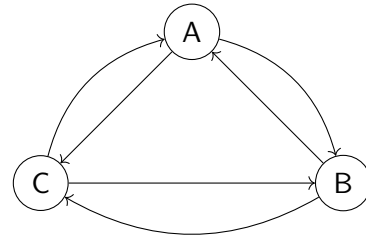
$$|E| = \sum_{i=1}^n (n-1) = n(n-1).$$

□

1.3.4 Example. Let us consider Figure 1.8a to be a complete undirected graph for $n = 4$, K_4 , and Figure 1.8b to be a complete directed graph for $n = 3$, K_3 .



(a) Complete undirected graph.



(b) Complete directed graph.

In this example we can say:

- For an undirected graph, the number of vertices is $n = 4$ and the number of edges is 6 using the formula in the above lemma.
- For a directed graph, the number of vertices is $n = 3$, and the number of edges is 6 using the formula above, again.

1.3.5 Path

1.3.6 Definition. A path in a graph is a sequence of nodes where each node in the sequence is connected to the next by an edge, and no node appears more than once in the sequence.

If we traverse a path, no vertices and edges are repeated.

1.3.7 Example. Let G be an undirected graph.

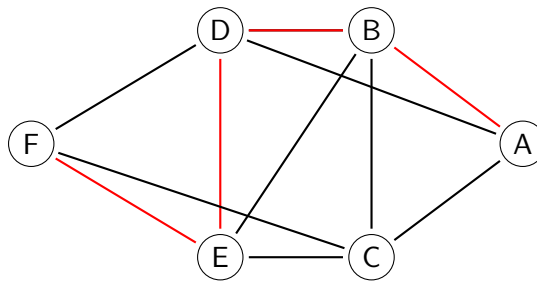


Figure 1.9: A Path from vertex A to vertex F .

In Figure 1.9, the red color is a path.

1.3.8 Connected Graph

1.3.9 Definition. A graph is connected if, for every pair of vertices, a path exists between the two vertices (Bondy and Murty, 1976).

1.3.10 Example. In Figure 1.10, a graph in which all vertices are connected.

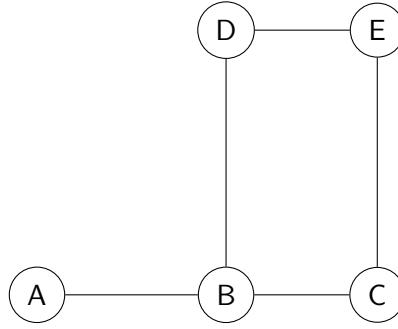


Figure 1.10: Connected graph.

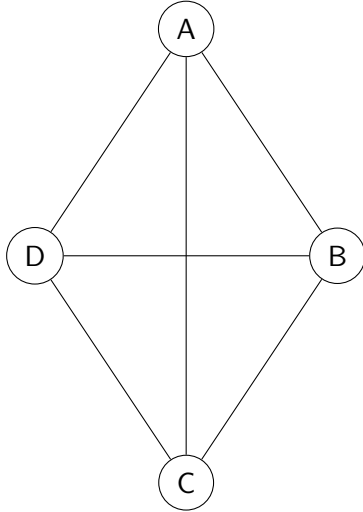
1.3.11 Subgraph of a Graph

Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be graphs.

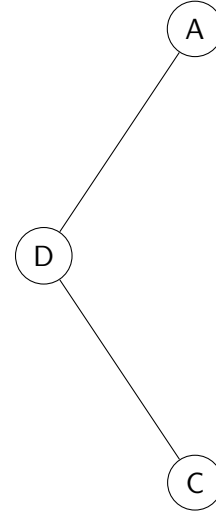
1.3.12 Definition. We say that $G_1 = (V_1, E_1)$ is a subgraph of $G_2 = (V_2, E_2)$ if and only if $V_1 \subseteq V_2$ and $E_1 \subseteq E_2$ (Bondy and Murty, 1976).

1.3.13 Example. In this example, Figure 1.11a has set of vertices $V_2 = \{A, B, C, D\}$, and set of edges $E_2 = \{\{C, A\}, \{C, B\}, \{C, D\}, \{B, A\}, \{B, D\}, \{D, A\}\}$.

And Figure 1.11b has set of vertices $V_1 = \{A, C, D\}$, and set of edges $E_1 = \{\{C, D\}, \{D, A\}\}$.



(a) Graph G_2 .



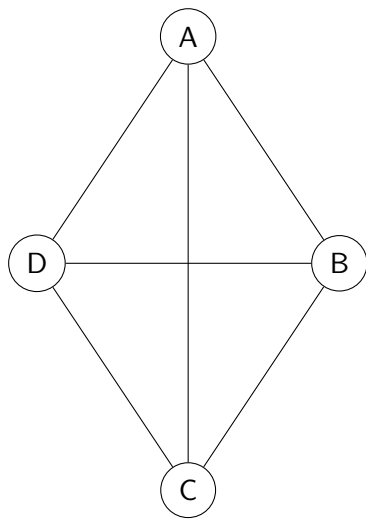
(b) Graph G_1 subgraph of G_2 .

1.3.14 Induced Subgraph

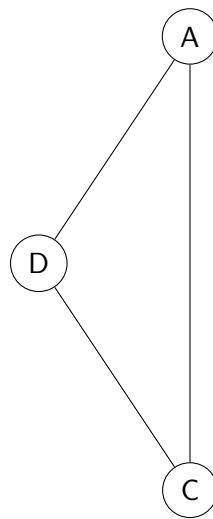
1.3.15 Definition. Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be graphs. G_1 is an induced subgraph of G_2 if it satisfies $V_1 \subseteq V_2$ and $E_1 = \{uv \in E_2 : u, v \in V_1\}$. An induced subgraph G_1 of a graph G_2 is a subgraph in which all edges between vertices of G_1 in G_2 are present.

1.3.16 Example. In this example, Figure 1.12a is G_2 and the Figure 1.12b is G_1 .

$V_2 = \{A, B, C, D\}$ and $V_1 = \{A, C, D\}$.



(a) Graph G_2 .



(b) Graph G_1 induced subgraph of graph G_2 .

1.3.17 Euler Path

1.3.18 Definition. An Euler Path is a path that traverses all edges exactly once. It doesn't have to return to the same vertex.

1.3.19 Example. Given a graph that has 5 vertices.

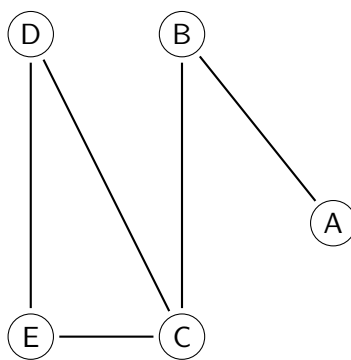


Figure 1.13: A graph on 5 vertices.

Figure 1.13 has an Euler Path given by $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow C$.

1.3.20 Cycle graph

1.3.21 Definition. A cycle in a graph is a list of vertices $v_1, v_2, \dots, v_n, v_1$ such that v_i is adjacent to v_{i+1} for all $1 \leq i \leq n-1$, v_n is adjacent to v_1 and no vertices are repeated. In addition, an Euler cycle is an Euler Path such that the first *vertex* is joined directly to the last *vertex* (Bondy and Murty, 1976).

The cycle graph with n vertices is denoted by C_n , such that:

- $V = \{v_1, v_2, \dots, v_n\}$.
- $E = \{\{v_i, v_{i+1}\} | 1 \leq i \leq n-1\} \cup \{v_1, v_n\}$.

1.3.22 Example. Figure 1.14 is an example of graph C_4 .

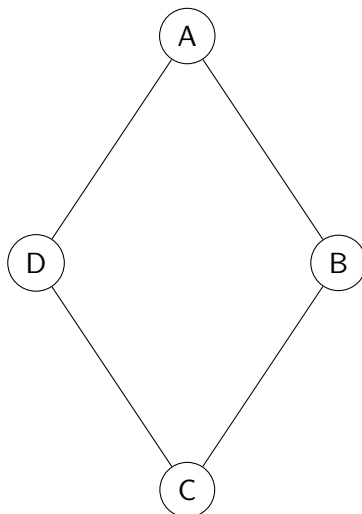


Figure 1.14: Cycle with 4 vertices.

1.3.23 Cyclic Graph

1.3.24 Definition. A graph is cyclic if it contains a cycle as a subgraph.

1.3.25 Example. Figure 1.15 is an example of a cyclic graph.

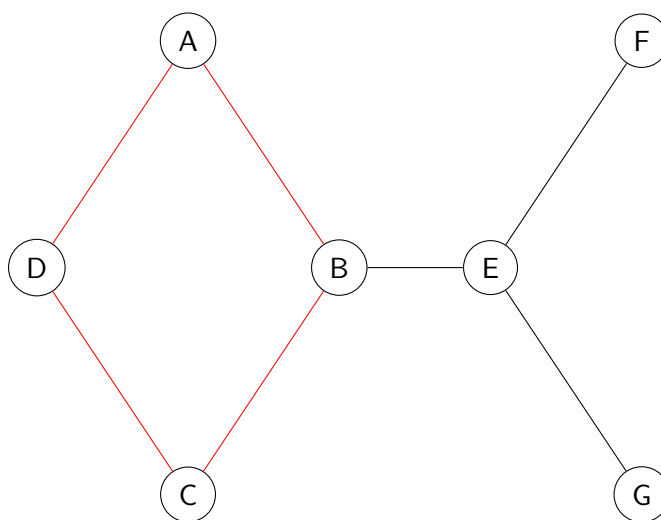


Figure 1.15: A cyclic graph.

In Figure 1.15, a red color means a cycle.

1.3.26 Definition. A circuit is a sequence of adjacent vertices that begins and ends at the same vertex. A circuit never repeats edges.

1.3.27 Example. Given a graph G be a directed graph. Figure 1.16 has an example of a circuit starting vertex and ending at the vertex A . So a circuit is $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow C \rightarrow B \rightarrow A$.

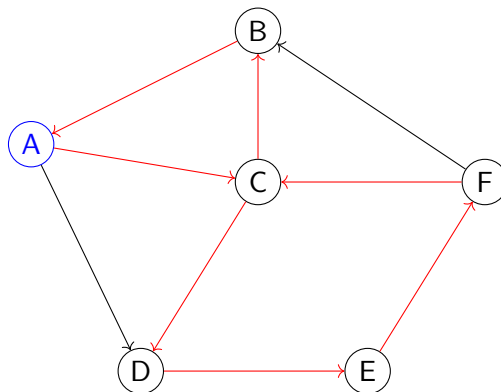


Figure 1.16: Circuit.

1.3.28 Remark. There is a difference between “cycle” and “circuit”:

We use “cycle” in an undirected graph, and “circuit” as an oriented cycle in a directed graph.

1.3.29 Definition. An Euler circuit is a circuit that uses every edge in a graph with no repetition. It must start and end at the same vertex.

1.3.30 Example. Figure 1.17 is an example of an Euler circuit.

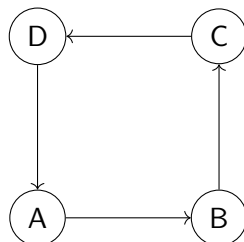


Figure 1.17: An Euler circuit that has 4 vertices.

Let us see this graph:

- If we draw this graph starting from vertex D we are possibly arriving at vertex D once.
- We can do it with another vertex.

1.3.31 Proposition. A graph has an Euler circuit, if and only if it is connected and each vertex has an even degree.

If a graph has a vertex with an odd degree, no Euler circuit is possible.

1.3.32 Example. Consider Figure 1.18.

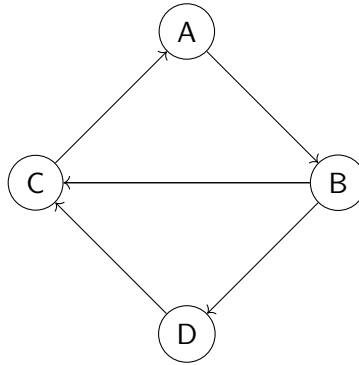


Figure 1.18: Directed graph G that has in-degree and out-degree.

In this graph, there is a vertex that has an odd degree, which means there is an odd number of edges to this graph. So when you traverse these edges, you will always have one edge left, which means you can't return to the starting point without revisiting at least one edge.

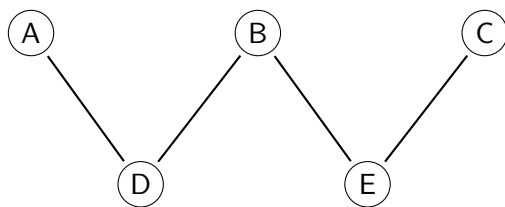
1.4 Special Graphs

1.4.1 Bipartite Graph

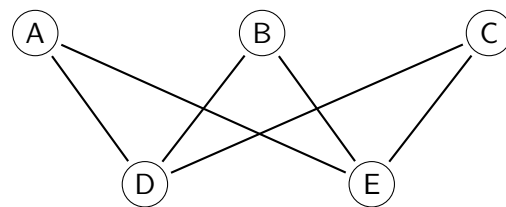
1.4.2 Definition. A graph $G = (V, E)$ is bipartite if the vertex set V can be partitioned into two sets V_1 and V_2 (the bipartition) such that (Bondy and Murty, 1976):

1. $V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$ (the subsets V_1 and V_2 are disjoint).
2. For every edge $uv \in E$ we have $u \in V_1$ and $v \in V_2$ or $u \in V_2$ and $v \in V_1$.
 - The partition $V = V_1 \cup V_2$ is called a bipartition of graph G .
 - If each vertex of V_1 is joined with each vertex V_2 , then the graph G is called complete bipartite graph.
 - If G is bipartite, it is usually denoted by $G = (V_1, V_2, E)$, where E is the set of edges.

1.4.3 Example. Figure 1.19a, and Figure 1.19b are bipartite graphs.



(a) Simple bipartite graph.



(b) Complete bipartite graph.

In Figure 1.19b, $V_1 = \{D, E\}$, and $V_2 = \{A, B, C\}$. All vertices from V_1 are joined to vertices V_2 , so we can say that, Figure 1.19b is a complete bipartite graph. In Figure 1.19a, $V_1 = \{A, B, C\}$, $V_2 = \{D, E\}$. So Figure 1.19a is a simple bipartite graph.

1.4.4 Trees

1.4.5 Definition. A tree is an undirected connected graph with no cycles.

A connected graph with n vertices is a tree if and only if it contains precisely $n - 1$ edges.

1.4.6 Example. Consider the tree in Figure 1.20.

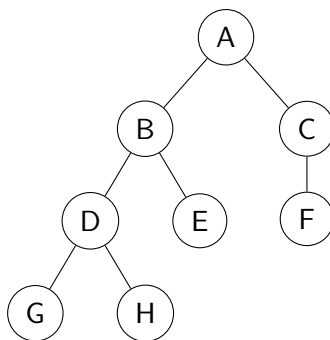


Figure 1.20: A tree.

In Figure 1.20, there are 8 vertices and 7 edges. So we can say that, Figure 1.20 is a tree.

In this chapter, we have discussed basic graphs including types of graphs, properties of graphs, and special graphs. So, in the next chapter, we will discuss optimization problems including the specific classes algorithm for graph optimization.

2. Optimisation

2.1 Graph Optimization Problems

Optimization involves finding the optimal solutions for allocations using a graph to satisfy certain objectives, such as minimization of costs and optimizing resource allocation. In this section, we discuss several such problems (Mark Needham, 2019).

2.1.1 Spanning tree

2.1.2 Definition. A spanning tree of a connected graph G is a subgraph of G that contains all vertices of G and is a tree.

The number of vertices $|V|$ in the graph and the spanning tree are the same, moreover, the spanning tree will have $|V| - 1$ edges.

Proof. Assume a graph has $|V|$ vertices and $|V|$ edges. If a graph has the same number of vertices and the number of edges, that means this graph creates a cycle because it is connected.

A spanning tree of a graph that $|V|$ vertices satisfies:

- At least it contains a tree that is a connected graph with no cycles.
- It has $|V| - 1$ edges.

Hence, a spanning tree guarantees for a graph that has $|V|$ vertices is connected with exactly $|V| - 1$ edges. $\square$

2.1.3 Example. Figure 2.1 shows an example of a weighted graph that has a 5 vertices.

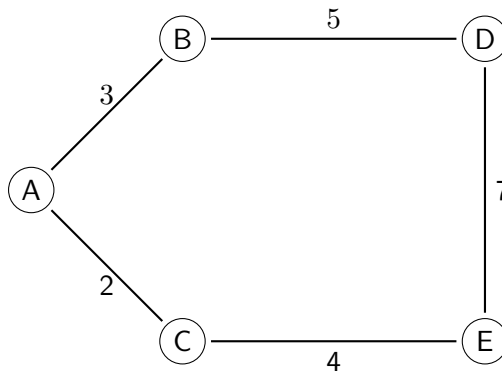
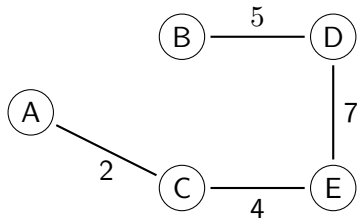
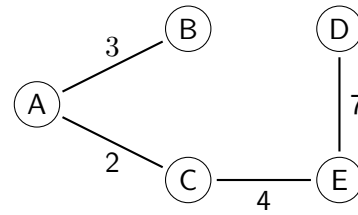


Figure 2.1: Undirected weighted graph G .

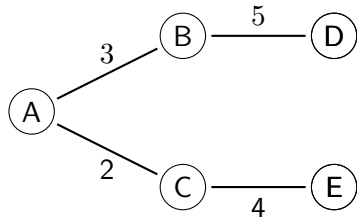
Here are spanning trees (ST) from Figure 2.1.



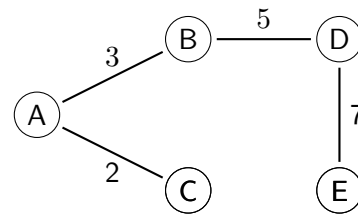
(a) Graph G with edge AB removed.



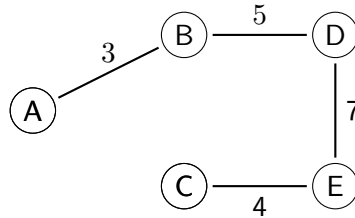
(b) Graph G with edge BD removed.



(c) Graph G with edge DE removed.



(d) Graph G with edge EC removed.



(e) Graph G with edge AC removed.

2.1.4 Minimum Spanning Tree (MST)

2.1.5 Definition. A minimum spanning tree (MST) is a tree with the minimum total weight (sum over all edge weights) among all the possible spanning trees.

Moreover, a complete undirected graph with n number of vertices can have n^{n-2} number of spanning trees according to Cayley's Formula (Diestel, 2017).

After drawing all spanning trees from Figure 2.1, we have Figure 2.2a with a total weight of 18, Figure 2.2b a total weight 16, Figure 2.2c a total weight 14, Figure 2.2d a total weight 17, Figure 2.2e a total weight 19.

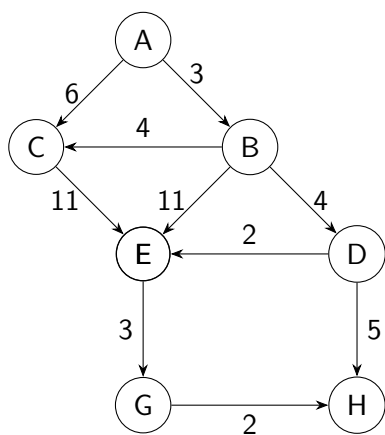
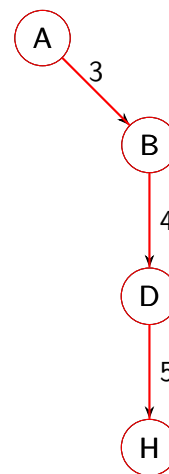
So from Figure 2.1, Figure 2.2c that has a total weight of 14 is a Minimum Spanning Tree (MST).

2.1.6 Shortest Path Problem:

The shortest Path Problem is an optimization problem that aims to find the shortest path from the source S of a vertex to other vertices of the graph.

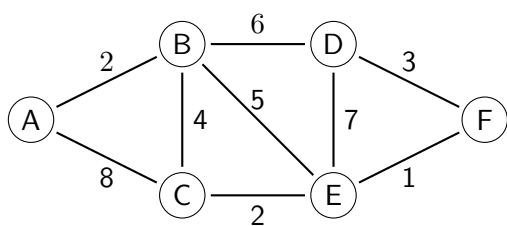
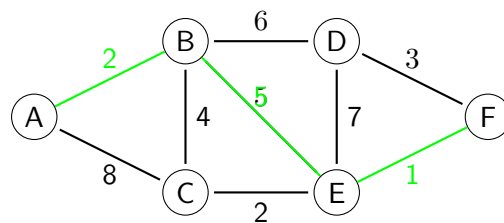
2.1.7 Definition. Given a weighted graph G , the shortest path in G from a source vertex to other vertices is a path from a source vertex to another vertex that has minimum weight (summed over all vertices in the path) (Bondy and Murty, 1976).

2.1.8 Example. Figure 2.3a is a weighted directed graph, and Figure 2.3b is the shortest path of Figure 2.3a from vertex A to vertex H .

(a) Directed Graph G (b) Shortest path from directed graph G

So, a path $A \rightarrow B \rightarrow D \rightarrow H$ with 12 total weights is the shortest path of Figure 2.3a.

2.1.9 Example. Figure 2.4a is an undirected graph and in Figure 2.4b a path $A \rightarrow B \rightarrow E \rightarrow F$ is the shortest path of the graph in Figure 2.4a from A to F .

(a) Undirected graph G (b) In green, a shortest path A to F in undirected graph G .

The green path is the shortest one with a length of 8.

2.2 Algorithms for Graph Optimization

Various algorithms and techniques can be used to tackle optimization problems efficiently in graph theory. But in the next section, let us discuss some prominent graph optimization algorithms.

2.2.1 Definition. A greedy algorithm is a type of optimization algorithm that makes locally optimal choices at each step to find a globally optimal solution.

2.2.2 Definition. Dynamic Programming (DP) is a method used in mathematics and computer science that can solve complex problems by breaking them down into simpler subproblems for which results are saved.

2.2.3 Definition. Given a weighted graph $G = (V, E)$. The Single-Source Shortest Path (*SSSP*) problem involves finding the shortest paths between a single source vertex $u \in V$ to other vertices $v \in V$ in the graph G .

In a weighted directed graph $G = (V, E)$, the shortest path problem finds the path with the lowest total weight between two vertices. Algorithms such as Dijkstra and Bellman-Ford solve this problem.

We have discussed minimum spanning trees, the shortest path problem, greedy algorithms, and dynamic programming. So, in the next chapter, we will explain some examples of algorithms that we can use to solve minimum spanning trees and shortest path problems.

In addition, we will discuss some algorithms like Bellman-Ford that can be used to solve various graph optimization problems, especially when the graph contains negative-weight edges.

3. Basic Algorithms

3.1 Algorithms for Minimum Spanning Trees (MST)

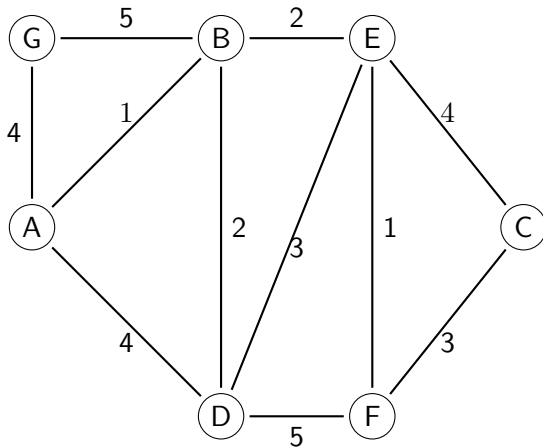
In this section, we will look at some commonly used graph algorithms for MST and provide an explanation and examples of each of them.

3.1.1 Kruskal's Algorithm

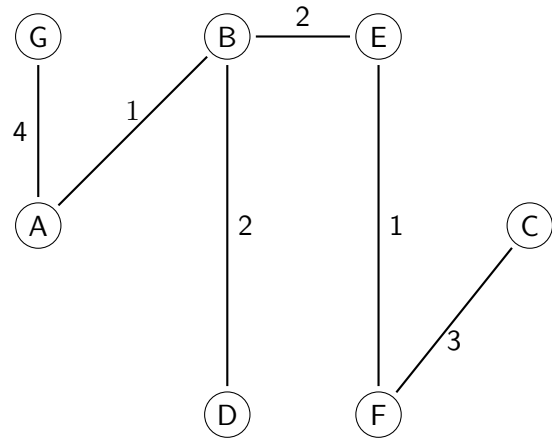
3.1.2 Definition. Kruskal's algorithm is an algorithm used to find a minimum spanning tree on an undirected graph $G = (V, E)$ that has positive weights (Wang et al., February 22nd, 2019b). The procedure is as follows:

1. Choose an edge that has a minimum weight.
2. Choose another edge that has the next minimum weight not create any cycle, and repeat this sequence until all edges and vertices are visited.
3. After using this algorithm, if a graph contains n vertices, it will have $n - 1$ edges.

3.1.3 Example. Let $G = (V, E)$ be a weighted graph.



(a) Graph G before using Kruskal's Algorithm.



(b) Graph G after Kruskal's Algorithm.

After following the steps to find an MST using Kruskal's Algorithm from a graph in Figure 3.1a, we have obtained a Figure 3.1b that has a total weight of 13. As we know, Kruskal's Algorithm is greedy, so the MST is 13.

3.1.4 Prim's Algorithm

3.1.5 Definition. Prim's algorithm is an algorithm to find a minimum spanning tree on a graph (Wang et al., February 22nd, 2019a). The procedure is as follows:

1. Select any vertex and choose the edge that has the smallest weight from a graph.

2. At each step, choose the edge with the smallest weight joining a vertex not yet visited, without forming a cycle.
3. Repeat steps 2, and continue this sequence until all vertices are visited.

3.1.6 Example. Let $G = (V, E)$ be a weighted undirected graph.

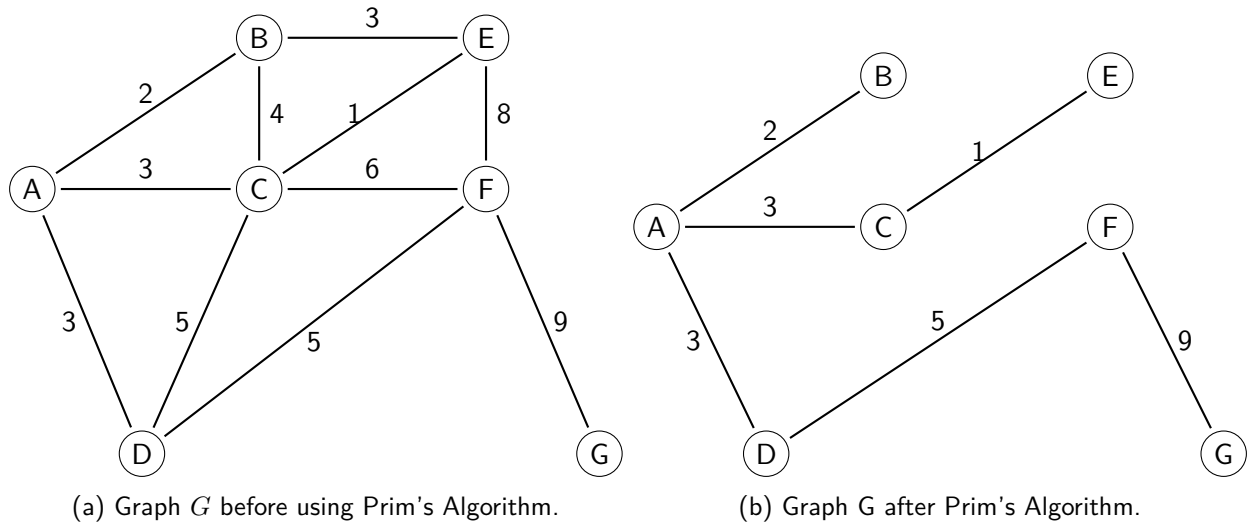


Figure 3.2: After using Prim's Algorithm in (a), we have obtained (b) that has a total weight of 23. So (b) is an MST from (a).

3.1.7 Remark. If you start with two different vertices, Prim's Algorithm may give two different results.

3.2 Algorithms for Single-source Shortest Path Problem (SSPP)

Here, we will look at some algorithms for single-source shortest-path problems and explain examples of each algorithm using these problems.

3.2.1 Dijkstra's Algorithm

3.2.2 Definition. Dijkstra's algorithm is a powerful method used to solve many problems involving the determination of the shortest path from a single source in a graph with non-negative edge weights (Bondy and Murty, 1976).

The procedure of Dijkstra is as follows:

1. Firstly, initialize the source vertex with a distance 0 and set the rest of the vertices to ∞ .
2. Secondly, visit the nearest vertex (the shortest distance away from the vertex to other vertices) not yet visited, examine it, update a distance if the new distance is found shorter than the previous distance, and mark the current vertex as visited.
3. In the end, repeat step 2 for the next nearest unvisited vertex until all vertices have been visited.

3.2.3 Remark. Dijkstra's Algorithm, we can work with either undirected or directed graphs (Gupta, CMU, Spring 2017).

Dijkstra's Algorithm is greedy, it may not work if there is a negative weight, but if the negative weights cycle exists, it fails because it can keep reducing the path weight indefinitely.

3.2.4 Example. Let Figure 3.3 be an undirected weighted graph.

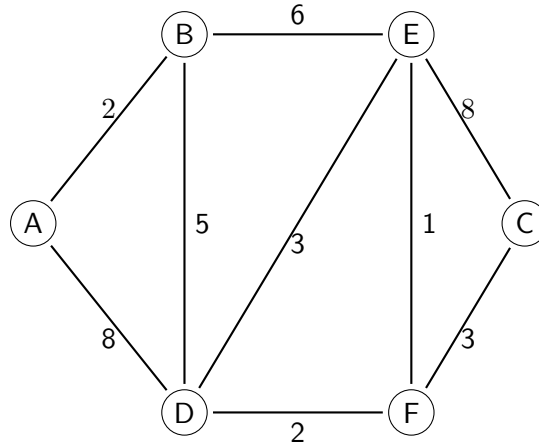


Figure 3.3: Undirected Graph G.

Dijkstra's formula

Here is a formula repeating in the second step of Dijkstra's Algorithm to facilitate understanding and implementation until all vertices are visited.

Let $G = (V, E)$ be a graph. Let $u \in V$ be a source and $v \in V$ is an unvisited vertex.

- $d(u)$: distance of the source or previous vertex, $d(v)$: distance of unvisited vertex.
- $c(u, v)$: means a weight of the edge between u and v .

$$\text{If } d(u) + c(u, v) < d(v) \text{ then we set} \\ d(v) = d(u) + c(u, v).$$

After following these rules and implementing this formula in Figure 3.3, we obtain Table 3.1:

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Table 3.1: Result for Dijkstra's algorithm in an undirected graph.

In Table 3.1 the red color means the previous node.

3.2.5 Example. Let Figure 3.4 be a directed weighted graph.

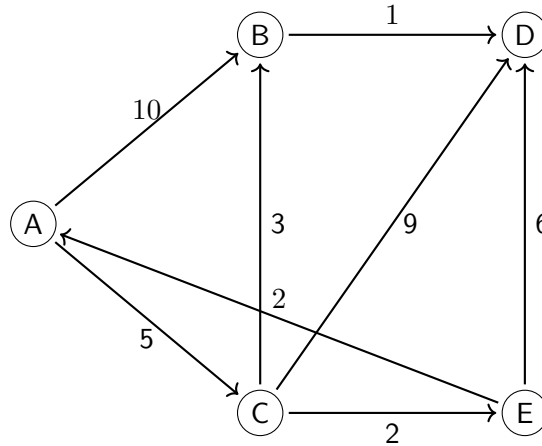


Figure 3.4: Directed Graph G.

We need to follow the rules and formula above to find the shortest path using Dijkstra's Algorithm.

So, after using and following step by step the rules and a condition of the formula of Dijkstra Algorithm in Figure 3.4, we get Table 3.2:

	A	B	C	D	E
A	0	∞	∞	∞	∞
C		10	5	∞	∞
E		8		14	7
B		8		13	
D			9		

Table 3.2: Result for Dijkstra for directed graph.

In Table 3.2 the red vertices mean the selected vertex and the green color means the short distance from the source vertex to another vertex.

3.2.6 Bellman-Ford Algorithm

3.2.7 Definition. Bellman-Ford algorithm is an algorithm to find the shortest path in the undirected or directed graphs that have both negative and positive edge weights (Gupta, CMU, Spring 2017). In the Bellman-Ford Algorithm, the procedure of steps is as follows:

1. Initialize all distance values as ∞ except source (always 0) and the choice of the source vertex is typically dictated by the problem we are trying to solve.
2. Do $n - 1$ times iterations if a graph has n vertices, and for each iteration, examine all edges.
3. Relaxing (visit all vertices). It means: that if $d(u) + c(u, v) < d(v)$ then we set $d(v) = d(u) + c(u, v)$, and finally, go on relaxing of all edges in $(n - 1)$ times when n is the number of vertices.

3.2.8 Example. Consider Figure 3.5 to be a directed graph.

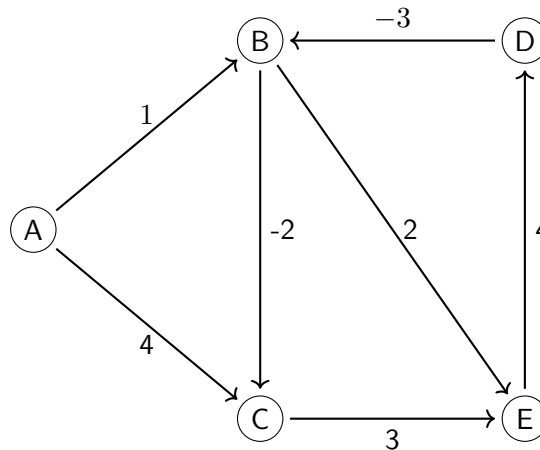


Figure 3.5: Directed Graph G that has negative edge weight.

Let's apply the steps of the Bellman-Ford Algorithm to find the shortest path in Figure 3.5.

In Figure 3.5, we consider a vertex A as the source that has distance 0 and we consider all other distances of the vertices to be ∞ .

Secondly, we examine all *edges* $\rightarrow \{(A, B), (A, C), (B, C), (B, E), (C, E), (E, D), (D, B)\}$.

Finally, we iterate 4 times in Figure 3.5 because we have 5 vertices. But if two or three successive iterations are the same, our iteration will be stopped and the algorithm will be finished.

	A	B	C	D	E
1 st step	0	∞	∞	∞	∞
1 st iteration	0	1	-1	2	6
2 nd iteration	0	1	-1	2	6
3 rd iteration	0	1	-1	2	6

Table 3.3: Result of Bellman-Ford Algorithm.

After iteration from Figure 3.5, Table 3.3, resume our steps and iteration. In this case, we found 3 iterations successively are the same. So we can say that the algorithm is finished.

In conclusion, the distances of each vertex are: $A \rightarrow \{0\}$, $B \rightarrow \{1\}$, $C \rightarrow \{-1\}$, $D \rightarrow \{2\}$, $E \rightarrow \{6\}$.

3.2.9 Drawbacks of the Bellman-Ford Algorithm

3.2.10 Lemma. Bellman-Ford Algorithm can't satisfy a correct answer if there is a negative cycle in a graph.

Proof. A negative weight cycle means a weighted graph that contains the negative weight, and if you add a part of the cycle of this graph, the total becomes negative. $\square$

3.2.11 Example. Consider Figure 3.6 to be a directed graph.

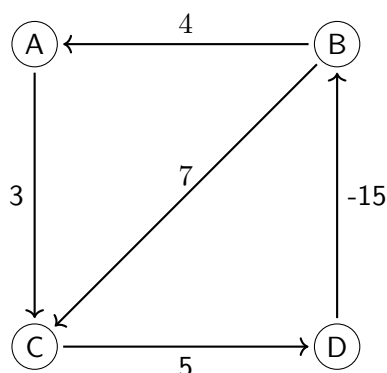


Figure 3.6: Directed Graph G that has a negative cycle.

To find the shortest path of Figure 3.6, we need to follow all the rules and steps in the Bellman-Ford Algorithm. After that, we analyze the result.

After analyzing the result, we see that there is a negative cycle in Figure 3.6.

A negative cycle is a cycle $D \rightarrow B \rightarrow C \rightarrow D$ that has the total weights: $7 + 5 + (-15) = -3$.

As we know, the Bellman-Ford Algorithm doesn't produce a correct answer if there is a negative cycle.

So, in Figure 3.6, the Bellman-Ford Algorithm can't satisfy a correct answer.

After discussing some algorithms to find minimum spanning trees and shortest path problems, in the next chapter, let's find the network flow and the minimum cut.

4. Network flows

A network is represented by a weighted graph in which, vertices represent stations, edges represent lines through which the given weight assigned to each edge shows the capacity (Righini, 2023b). Capacity is the maximum amount of flow possible per unit of time through that line.

4.0.1 Definition of Network (as an edge-weighted Directed Graph)

A network is defined as a directed weighted graph $G = (V, E)$ with two distinguished points s called the source and t called the sink (destination) (Righini, 2023a). Moreover, all edges $(x, y) \in E$ have a nonnegative capacity $c(x, y) \geq 0$.

4.0.2 Definition of Flow and Importance of Network Flows in various applications (transportation, telecommunications, etc.)

4.0.3 Definition. Let $G = (V, E)$ be a network, where V vertices and E edges. A flow is a function $f : V \times V \rightarrow \mathbb{R}$ satisfying (Jurcik, Accessed May 2024):

- $\forall x, y \in V \implies \boxed{f(x, y) \leq c(x, y)}$ (capacity constraints).
- $\forall x \in V - \{s, t\} \implies \boxed{\sum_{y \in V} f(x, y) = \sum_{y \in V} f(y, x) = 0}$ (flow conservation).

4.1 Maximum Flow Problem

The maximum flow problem is an optimization problem in graph theory that involves finding the maximum amount of flow that can be sent through a network.

4.1.1 Example. Figure 4.1 represents a flow network where every edge(arc) has a capacity.

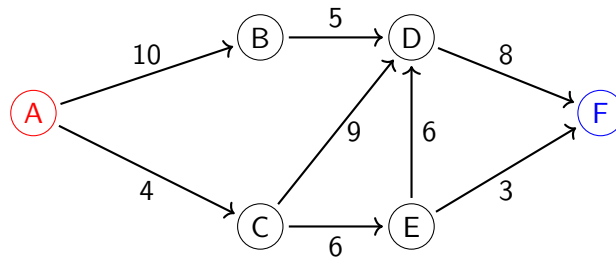
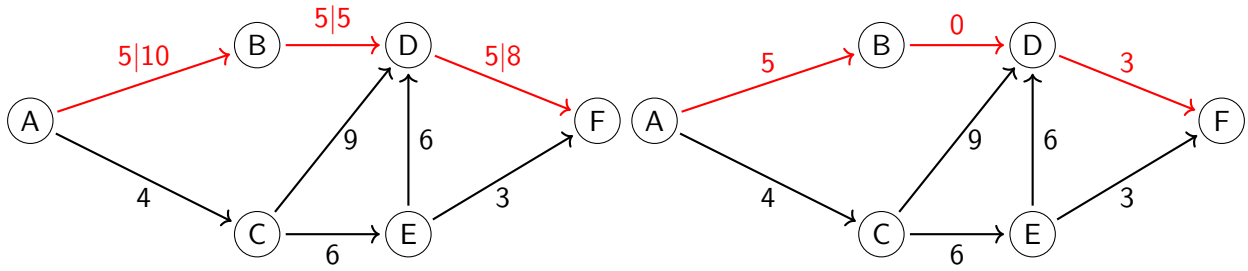


Figure 4.1: Maximum Flow Problem.

Maximum Flow Problem Steps (Righini, 2023a). Here are the steps for finding the maximum flow:

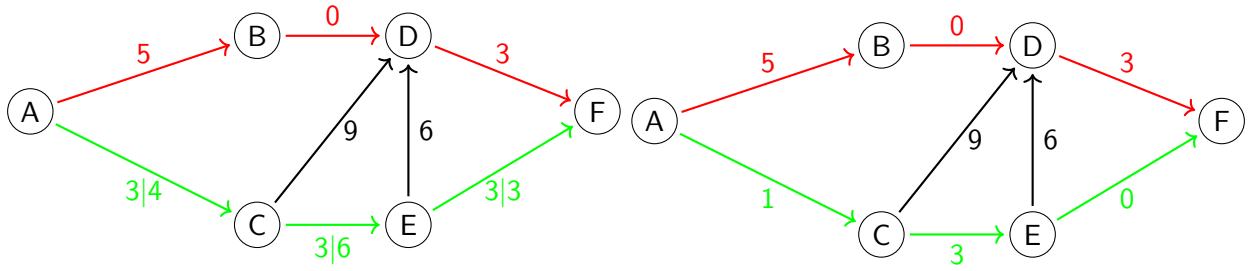
- Choose an arbitrary source and put a sink;
- From a source to sink, choose a minimum capacity, it is called a flow, and subtract all other capacities in the same path that you have chosen by a flow.
- Repeat the second step until all paths from the source to the sink are visited.
- At the end, add all flows from each path that we have visited.

4.1.2 Example. In Figure 5.1, we choose the vertex A as a source and the vertex F as a sink(destination).



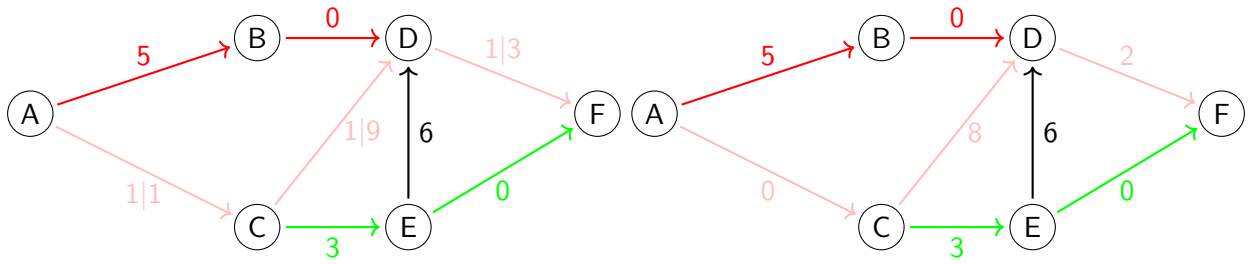
(a) First iteration for maximum flow, $flow = 5$.

(b) Network obtained after first iteration.



(c) Second iteration for maximum flow, $flow = 3$.

(d) Network obtained after the second iteration.



(e) Fourth iteration for maximum flow, $flow = 1$.

(f) Final network for Maximum Flow.

So, the maximum flow given by the graph G is $5 + 3 + 1 = 9$.

4.1.3 Formulation of the Maximum Flow Problem – Applications of Maximum Flow

Given a Network digraph $G = (V, E)$ has a source $s \in V$, a sink $t \in V$, and edge capacity $c : E \rightarrow \mathbb{R}^+ \cup \{0\}$ (Righini, 2023a).

- $\forall (x, y) \in E, 0 \leq f(x, y) \leq c(x, y)$.
- $\forall y \in V - \{s, t\}, \sum_{x \in V} f(x, y) = \sum_{z \in V} f(y, z)$.
- The value of a flow: $|f| = \sum_{y \in V} f(s, y) = f(s, V) = f(V, t)$.

4.1.4 Ford-Fulkerson Algorithm (Augmenting Paths, Residual Graph)

The Ford-Fulkerson Algorithm is a greedy approach algorithm to calculate the maximum flow in the network.

4.1.5 Terminologies

- **Augmenting Paths:** It's a path from the source to the sink.
- **Residual Graph:** It's a graph that indicates additional possible flow (Righini, 2023a).

Here are the following simple ideas of the Ford-Fulkerson Algorithm.

- Initialize a flow as 0;
- While the augmenting path from source to sink exists; add this path flow to flow until no augmenting path from source to sink;
- Finally, add the path flow to the total.

4.1.6 Example. Given a network $G = (V, E)$ with a source s and a sink t .

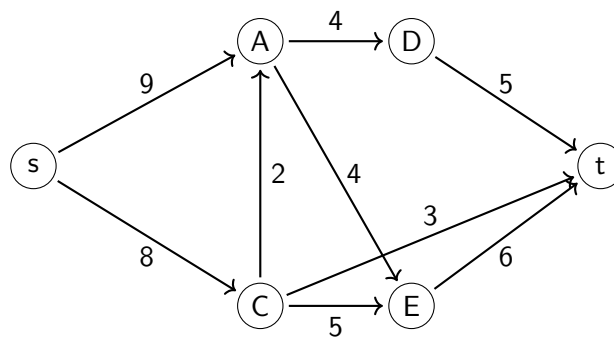


Figure 4.3: Network G .

Table 4.1 resumes all steps of Ford-Fulkerson until no augmenting path from a source s to a sink t of a network G a part of an initialization.

Augmenting Paths	Flow
Step 1: $s \rightarrow A \rightarrow D \rightarrow t$	4
Step 2: $s \rightarrow A \rightarrow E \rightarrow t$	4
Step 3: $s \rightarrow C \rightarrow E \rightarrow t$	2
Step 4: $s \rightarrow C \rightarrow t$	3

Table 4.1: Ford-Fulkerson.

4.1.7 Edmonds-Karp Algorithm

There are the steps for the Edmonds-Karp Algorithm (Klau, 2008).

- Step 1: Select any path from the source to the sink;
- Step 2: Choose an edge that has minimum weight and update;
- Step 3: Repeat step 2 until all augmenting path in a network are break;
- Step 4: Finally, add path flow to the total.

4.1.8 Remark. During the Edmonds-Karp Algorithm works, the edge with 0 capacity will be ignored in the next iteration, we could remove it as it has 0 capacity, and we'll keep it as a placeholder. Moreover, the Edmonds-Karp Algorithm is identical to the Fulkerson-Algorithm.

4.1.9 Example. Consider $G = (V, E)$ that has a source and a sink.

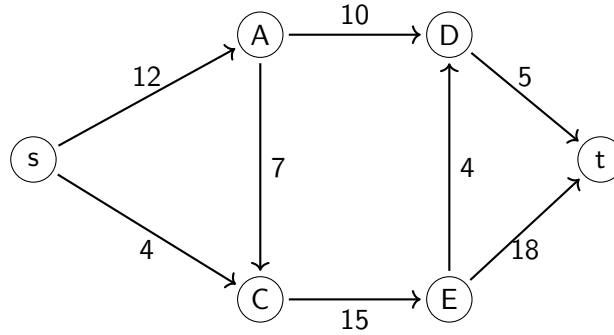


Figure 4.4: Network G .

Table 4.2 resumes the steps of the Edmonds-Karp Algorithm starting from the source s to the sink t .

Augmenting Paths	Flow
Step 1: $s \rightarrow A \rightarrow D \rightarrow t$	5
Step 2: $s \rightarrow A \rightarrow C \rightarrow E \rightarrow t$	4
Step 3: $s \rightarrow A \rightarrow C \rightarrow E \rightarrow t$	7

Table 4.2: Edmonds-Karp.

4.2 Minimum Cut Problem

4.2.1 Cut

4.2.2 Definition. Given a flow network $G = (V, E)$ that has a source S and the sink T . A cut (S, T) of flow network G is a disconnection in two sets X and Y of paths connected from the source to the sink, where $S \in X$ and $T \in Y$ (Jurcik, Accessed May 2024).

4.2.3 Minimum Cut Problem

4.2.4 Definition. The Minimum Cut Problem (or min-cut problem) of a network G that has a source S and a sink T , is to find the cut of the network G that has a minimum capacity (Sidiropoulos, 2017).

4.2.5 Formulation of the Minimum Cut Problem – Applications of Minimum Cut

Given a network $(G = (V, E), s, t, c)$, $V = S \cup T$, $S \cap T = \emptyset$, with $s \in S$ and $t \in T$.

- The flow across the cut: $f(S) = \sum_{x \in S} \sum_{y \in T} f(x, y) - \sum_{y \in S} \sum_{x \in T} f(y, x)$.
- The capacity of the cut: $C(S) = \sum_{x \in S} \sum_{y \in T} c(x, y)$. The capacity of cut (S, T) = sum of weights leaving S .

- The capacity constrain: $f(S) \leq C(S)$.

4.2.6 Relationship between Maximum Flow and Minimum Cut

As we know, in the maximum flow, we update the capacity that has a minimum value of each augmenting path in the network from the source to the sink, and we add that if there is no augmenting path from the source to the sink in the graph G (Klau, 2008). The total of that capacity from each iteration, we call that a Maximum Flow. In the Minimum Cut, we find the cut that has a minimum capacity and it is called Minimum Cut.

In this case, the values of Maximum Flow and Minimum Cut are the same.

If Maximum Flow = Minimum Cut:

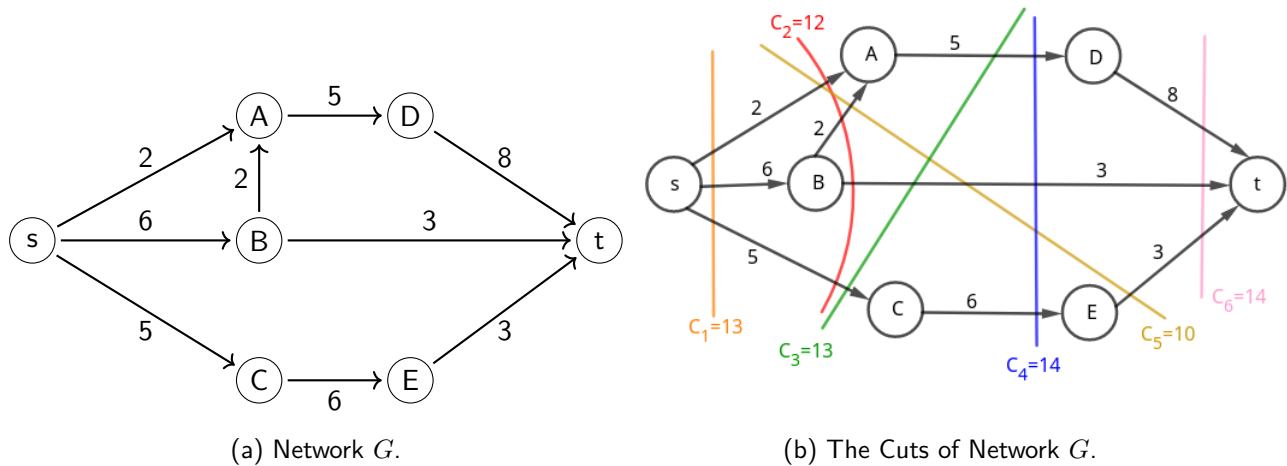
- The flow is maximal.
- The cut is minimal.

4.2.7 Finding the Minimum Cut

According to the Definition 4.2.3, here are the steps to find the Minimum Cut of a network G from the source S to the sink T (Goemans, Accessed May 2024).

- Firstly, find all possible cuts in a network G from the source to the sink;
- Secondly, add the capacities of each cut obtained;
- Finally, choose a cut that has the minimum value capacity.

4.2.8 Example. In this example, we find the Minimum Cut of the Network G following the steps for finding the Minimum Cut of the network.



After following all the steps for finding the minimum cut, we obtain $C_5 = 10$.

5. Implementation of the algorithm in Python, and comparison of the results from Google Maps

5.1 Algorithms Implementation

Here we have chosen Cape Town, a city in South Africa. We used two algorithms: Dijkstra's Algorithm and the Bellman-Ford Algorithm to find the shortest path between the two locations in Cape Town. In addition, the results depend on the data that we have selected from Google Maps and the structure of the graph from this data.

5.1.1 Dijkstra's Algorithm

We used Dijkstra's Algorithm to illustrate the shortest path between two locations in Cape Town. During the implementation, we choose the sources and destinations, and we have different results depending on the sources and destinations of locations that we need. As we know, Dijkstra's Algorithm works both on undirected and directed graphs. Firstly, if our data has the form of an undirected graph, our results from the source and the destination give the same result as from the destination to the source. If we select two locations that have a 100 kilometers distance, for example from *Muizenberg* to *Stellenbosch*, it is the same as from *Stellenbosch* to *Muizenberg*.

Secondly, if our data has a directed graph form, the distance between two locations may be different.

5.1.2 Example. From the implementation in Python of the data using Dijkstra's Algorithm, we have obtained the output by plotting these data.

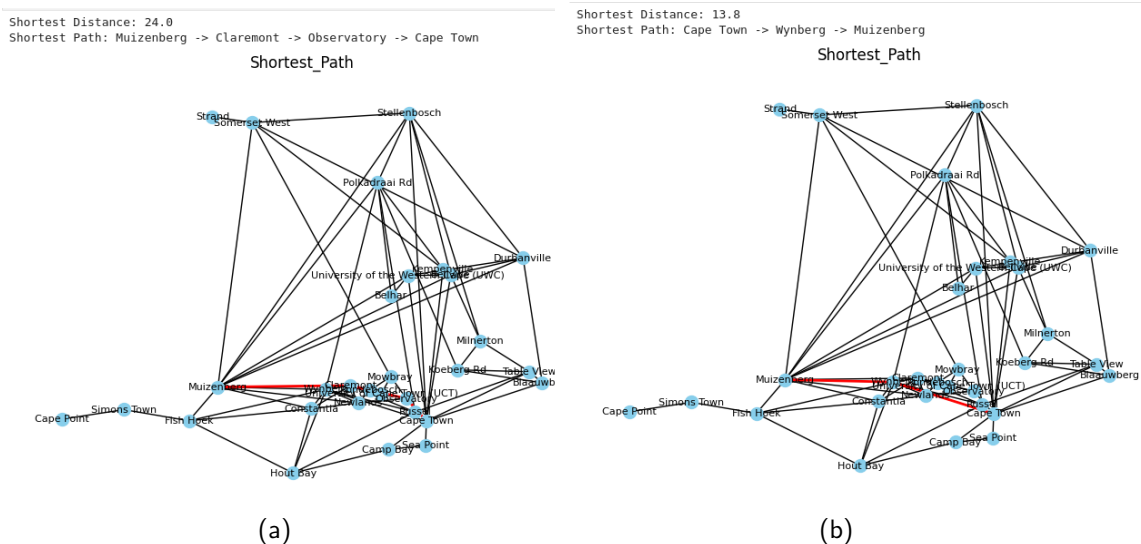


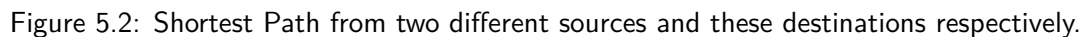
Figure 5.1: Shortest Path from each source to their destination.

From Figure 5.1, we have had two different results such as:

- (a) the result from Muizenberg.

- We chose the source as Muizenberg, and the destination is Cape Town. Using Dijkstra's Algorithm, we have the shortest path: *Muizenberg* $\rightarrow$ *Claremont* $\rightarrow$ *Observatory* $\rightarrow$ *Cape Town* with distance 24 km.

5.1.3 Example. To make sure the data that we have created has the form of an undirected or directed graph, we choose two locations again. Here we choose two locations such as Strand and Hout Bay from the data.



In Figure 5.2, the shortest path from Strand to Hout Bay is red color in (a) such as *Strand* $\rightarrow$ *Somerest West* $\rightarrow$ *Stellenbosch* $\rightarrow$ *Polkadraai Rd* $\rightarrow$ *Russel* $\rightarrow$ *Hout Bay* with distance 64,7 km. And from Hout Bay to Strand is red color in (b) such as *Hout Bay* $\rightarrow$ *Wynberg* $\rightarrow$ *Clermont* $\rightarrow$ *Rondebosch* $\rightarrow$ *Mowbray* $\rightarrow$ *Somerset West* $\rightarrow$ *Strand* with distance 67,3 km. According to the different results from each example, we can say that the data that we have got from *OpenStreetMap(OSM)* has a directed graph form.

As we know, the Bellman-Ford Algorithm works both on undirected and directed graphs. Referring to an explanation in Dijkstra, we are working on data that has the form of a directed graph. We have implemented that data in Python using the Bellman-Ford Algorithm, choosing different sources from this data, the algorithm works well and different results are obtained depending on the source that we have chosen. It gives the shortest path from each location that we have selected. Here is an example from the implementation of the Bellman-Ford Algorithm using Python.

5.1.5 Example. We choose two different source locations from the data such as Muizenberg and Cape Town.

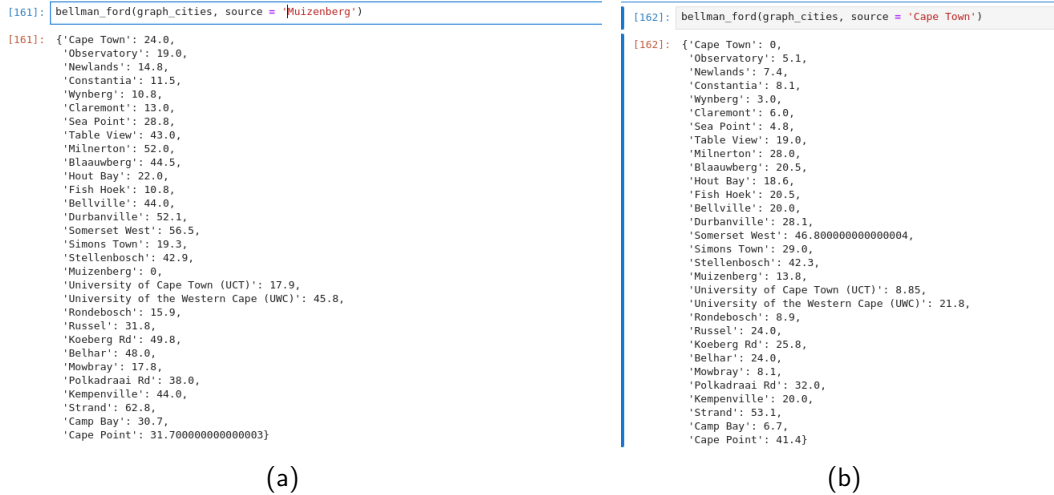


Figure 5.3: Shortest Path from two sources such as Muizenberg and Cape Town.

Figure 5.3 resumes the result from each source that we have chosen during the implementation following:

- (a) all shortest paths of each location from the source Muizenberg.
- (b) the shortest path from the source Cape Town.

From the implementation of the Bellman-Ford Algorithm in Python choosing two sources such as Muizenberg and Cape Town, our output like source *Muizenberg* has the shortest path *Cape Town* $\rightarrow$ {24 km} and source *Cape Town* has the shortest path *Muizenberg* $\rightarrow$ {13,8 km}, combining with others the shortest path of each location in a data.

5.1.6 Example. We have implemented again the Bellman-Ford Algorithm in two source locations such as Strand and Hout Bay and obtained the results here.

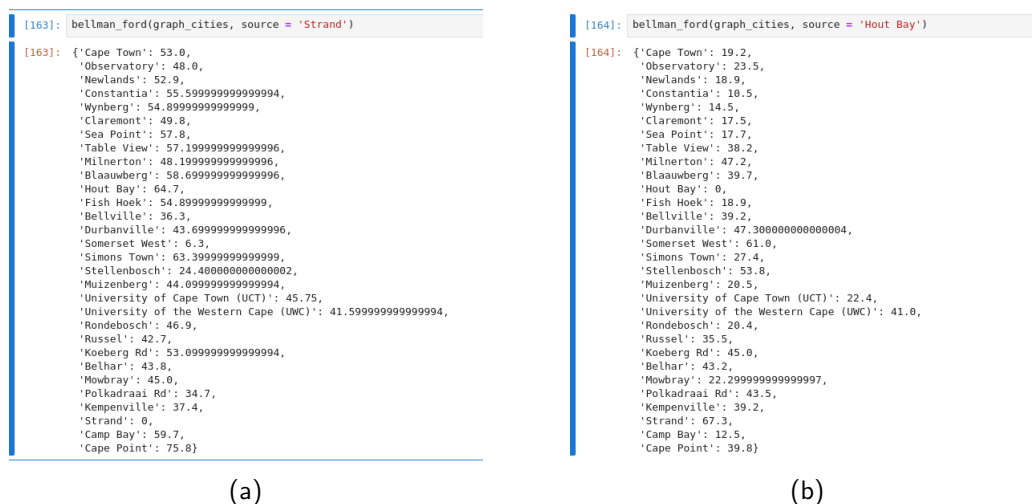


Figure 5.4: Shortest Path from two different locations.

In Figure 5.4, we have two locations such as Strand and Hout Bay, and their results are:

- (a) the result from the source in Strand.
- (b) the output from the source in Hout Bay.

For each source that we have chosen from data and using Bellman-Ford for implementation in Python, we have the result same as *Strand* has the shortest path *Hout Bay* $\rightarrow \{64, 7 \text{ km}\}$ and source *Hout Bay* has the shortest path *Strand* $\rightarrow \{67, 3 \text{ km}\}$, with others the shortest path of each location in a data.

We observe that the results after implementation in Python for each algorithm such as Dijkstra's Algorithm and Bellman-Ford Algorithm, are the same. Generally, we can say that all the shortest paths from Dijkstra's Algorithm are the same as those from the Bellman-Ford Algorithm.

5.2 Comparison with Google Maps

5.2.1 The shortest paths generated by algorithms with those provided by Google Maps

After implementing data in Python with two algorithms, we have had the results from these algorithms be the same. Using Google Maps, the shortest path is approximately the same as from these algorithms.

To make clear the comparison, we take two different examples from the algorithm and two from Google Maps.

5.2.2 Example. Here is the shortest path from Cape Point to Durbanville using algorithms and Google Maps.

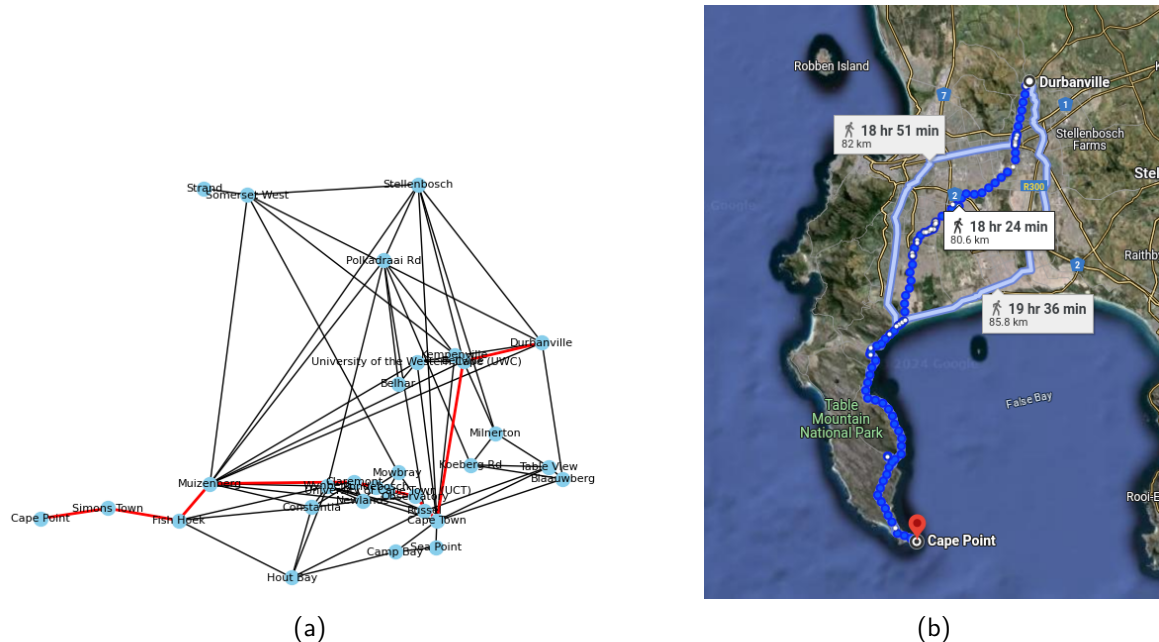


Figure 5.5: Shortest Path from Cape Point to Durbanville using Code and Google Maps.

- From Figure 5.5, in (a) the shortest path starting at Cape Point and ending at Durbanville is red color following *Cape Point* $\rightarrow$ *Simon's Town* $\rightarrow$ *Fish Hoek* $\rightarrow$ *Muizenberg* $\rightarrow$ *Claremont* $\rightarrow$

Observatory → Cape Town → Bellville → Durbanville, with distance 83,8 km.

- From Figure 5.5, Google Maps in (b), we have 3 different ways from Cape Point to Durbanville, and the blue color means the shortest path between these locations, with distance 80.6 km.

5.2.3 Example. Here is the shortest path from Cape Town to Stellenbosch after implementation in Python and from Google Maps.

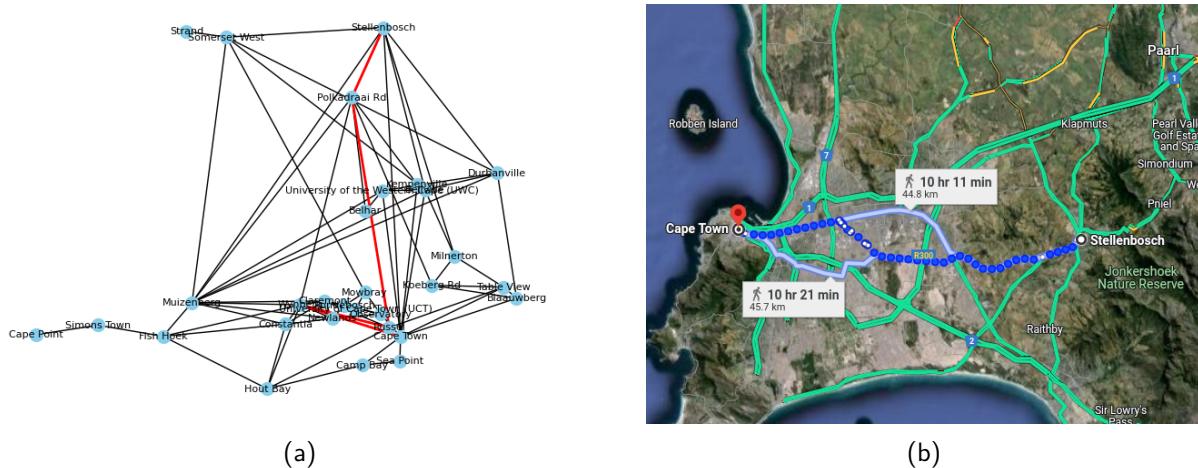


Figure 5.6: Shortest Path starting at Cape Town and ending at Stellenbosch from Code and Google Maps.

- In Figure 5.6, we have selected two locations such as Cape Town and Stellenbosch. We have implemented it in Python using the algorithm with data choosing the source as Cape Town and the destination as Stellenbosch. The red color is the shortest path from Cape Town to Stellenbosch following the locations such as *Cape Town → Wynberg → Russel → Polkadraai Rd → Stellenbosch*, with distance 42,3 km.
- Looking from Google Maps, we see 3 different paths but the shortest path of them is the blue color starting from Cape Town and ending to Stellenbosch with distance 44,5 km.

5.2.4 Analysis of discrepancies

Dijkstra's Algorithm and Bellman-Ford Algorithm are used to find the shortest path problem. The results from these algorithms are not effectively the same as those from Google Maps.

- For the algorithms, we usually find the shortest path by considering the distance defined on the graph or data.
- Google Maps, the user preference can be based on the road with less traffic, shorter distance, and type of road.

5.2.5 The practicality and real-world usage of these algorithms in services like Google Maps

As we know, graph theory and algorithms are important to the service of Google Maps.

- **Practicality:** Working in algorithms like Dijkstra's Algorithm and Bellman-Ford Algorithm for finding the shortest path between two locations. These algorithms work on a weighted graph,

such that, in a graph, the vertices represent intersections or locations, the edges represent the roads, and the weights represent the distances or the duration.

- **Real-World usage:** Google Maps is used for finding the shortest path between two locations same as Dijkstra and Bellman-Ford. It is faster by evaluating different possible paths and giving directly the shortest path.

They are used by Uber's driver if there is a person who checks from any location, the driver finds the shortest path from his position to the person who checked him.

5.3 Performance Analysis

5.3.1 Time Complexity

Time complexity is a method to analyze the running time of algorithms. In our case, we work on two algorithms such as Dijkstra's Algorithm and Bellman-Ford Algorithm. Now, we are going to analyze and discuss the best and worst-case time complexities of these algorithms. Here is a short explanation of the time complexity of each algorithm.

- Dijkstra's Algorithm, the complexity time for implementation is $O((n+m) \log n)$, where n number of vertices and m number of edges in the graph.
- Bellman-Ford Algorithm, the complexity time of an implementation is given by: $O(n \times m)$, such as n number of vertices and m number of nodes.

Analyzing their complexity times, we take our data and generate approximately the number of vertices and nodes.

Dijkstra's Algorithm uses a priority queue to efficiently select and update the shortest path for each vertex, and as we know his time complexity is $O((n+m) \log n)$, while Bellman-Ford does this iteration relaxing all edges in $n - 1$ times with time complexity $O(n \times m)$. Hence Dijkstra's Algorithm generally has less time complexity than the Bellman-Ford Algorithm, given that $\log n$ is less than n ($\log n < n$).

After analyzing their running times, the Bellman-Ford Algorithm is more consuming time than the Dijkstra Algorithm.

So we can say that with a good implementation, the running time of the Dijkstra Algorithm is lower than that of the Bellman-Ford Algorithm due to its lower complexity compared to Bellman-Ford (Cormen et al., 2001).

The CSV file of the data and the Python code of **Dijkstra, Bellman-Ford** can be found [here](#).

6. Conclusion

Graph theory enables us to study many fields, and it solves some difficult problems, it gives us the approximate solutions to the problem. Its origins are back to the problems of the Seven Bridges of Königsberg by Leonard Euler in 1736.

Through graph theory, we can solve many problems using two types of algorithms, Greedy Algorithms and Dynamic Programming. We examine and think of different fundamental techniques for solving optimization problems such as finding the minimum spanning tree and the shortest path problem. Additionally, graph theory enables us to focus on network flows so that we can model again a graph optimization problem as a maximum flow problem and a minimum cut problem. For a maximum flow problem, we worked only on the directed graph using the Ford-Fulkerson Algorithm and Edmonds-Karl Algorithm to solve an optimization problem.

For focusing on the implementation of the algorithms in Python, like Dijkstra, and Bellman-Ford, we have obtained many results and discussed and analyzed the validation of each algorithm and its application in the real world.

Acknowledgements

First of all, I thank Prof. Neil Turok founder of the AIMS, and Prof. Ulrich Paquet, the Director of AIMS South Africa, for his role in setting up teaching in this great school.

Then, I warmly thank Dr. Karin-Therese Howell the Academic Director of the African Institute for Mathematical Sciences South Africa, Cape Town (AIMS South Africa).

I am particularly grateful to my supervisor, Dr. Audace A. V. Dossou-Olory teaching at the University of Abomey-Calavi, Benin, and Dr. Sarah J. Selkirk teaching at the University of Warwick, UK, for their support and wise advice throughout the production of this project.

I also thank all the teachers of AIMS for their dedication, as well as the members of the jury for agreeing to judge my work.

I also thank all tutors at AIMS for their morale and support during my study starting September 2023 until now June 2024.

Thank you to all my classmates and colleagues for their contribution, directly or indirectly, to the success of my study here at AIMS.

Finally, I thank my family and my parents who encouraged me for all my goals.

References

Bleiel, J. *Graphs Algorithms*. 2019.

Bondy, J. A. and Murty, U. S. R. *Graph Theory With Applications*. Macmillan, London, 1976. doi: 10.1007/978-1-349-03521-2.

Chartrand, G. and Lesniak, L. *Graphes and Digraphs*. 1979.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts London, England, 2 edition, 2001.

Diestel, R. *Graph Theory*. Springer, 5th edition, 2017. Cayley's Formula is discussed in detail in this book.

Goemans, M. X. 18.433: Combinatorial optimization. Course materials, Accessed May 2024. URL [URLofcoursewebpage](#).

Gupta, A. Shortest paths and seidel's algorithm, CMU, Spring 2017.

Jurcik, K. Operations research seminar: Maximum flow problem. Seminar presentation, Accessed May 2024. Available upon request from the speaker or seminar organizer.

Klau, G. W. Discrete mathematics for bioinformatics. Lecture notes, January 2008. Accessed on January 23, 2008, 17:21.

Korte, B. and Vygen, J. *Combinatorial Optimization: Theory and Algorithms*. Algorithms and Combinatorics. Springer Berlin Heidelberg, 2007. ISBN 9783540718444.

Korte, B. and Vygen, J. *Algorithms and Combinatorics*. Springer Fourth Edition, 2008.

Mark Needham, A. E. H. *Graph Algorithms: Practical Examples in Apache Spark and Neo4j*. O'Reilly Media, 1 edition, 2019. ISBN 1492047686, 978-1492047681.

Righini, G. Maximum flow problem (part i). Combinatorial Optimization, 2023a.

Righini, G. Maximum flow problem (part ii). Combinatorial Optimization, 2023b.

Sidiropoulos, A. Math 8500 algorithmic graph theory, spring 2017, osu: Lecture 1: Min-cut and k-cut. Lecture notes, scribed by Ario Salmasi, 2017. Available upon request from the instructor or course website.

Wang, M. L., Charles Zhao, E., and by Jongwan Kim, D. *Minimum Spanning Trees*. February 22nd, 2019a.

Wang, M. L., Charles Zhao, E., and by Jongwan Kim, D. *Minimum Spanning Trees*. February 22nd, 2019b.